

Andra strängrelaterade funktioner

Andra strängrelaterade funktioner kan man själv skapa genom att direkt manipulera strängars innehåll såsom vi gjort i våra exempelfunktioner. Det finns dock ett antal olika funktioner som gör det lättare att manipulera strängar. Se dock [avsnittet Mera information](#) för mera information om vilka funktioner som finns tillgängliga.

Söka tecken

Sökning av ett viss tecken i en C-sträng kan ganska enkelt implementeras genom att man jämför strängens tecken enskilt tills man hittat det sökta tecknet eller kommit till strängens slut. En enkel version kan skrivas på följande sätt:

```
#include <iostream>

int findChar (char * String, char Seek) {
    int Index = 0;

    // iterera tills vi kommer till slutet
    while ( String[Index] != '\0' ) {
        // är det korrekt tecken?
        if ( String[Index] == Seek ) {
            // vi har korrekt index
            return Index;
        }
        Index++;
    }

    // inget index hittades
    return -1;
}

int main () {
    int Index;
    char * S = "En sträng ur vilken vi söker text";

    cout << "Söksträngen är: '" << S << "'" << endl;

    // sök ett 'v'
    if ( ( Index = findChar (S, 'v' ) ) != -1 ) {
        cout << "Hittade ett 'v' på index " << Index << endl;
    }
    else {
        cout << "Hittade inget 'v'" << endl;
    }

    // sök ett 'P'
    if ( ( Index = findChar (S, 'P' ) ) != -1 ) {
        cout << "Hittade ett 'P' på index " << Index << endl;
    }
    else {
        cout << "Hittade inget 'P'" << endl;
    }
}
```

Funktionen `findChar()` söker igenom strängen `String` tills antingen slutet av strängen nås eller korrekt tecken hittats. Om ett tecken hittas returneras dess index, men om inget tecken motsvarande `Seek` hittas returneras `-1` för att indikera en misslyckad sökning. Körning av programmet ger:

```
% ./CStringFind1
Söksträngen är: 'En sträng ur vilken vi söker text'
Hittade ett 'v' på index 13
Hittade inget 'P'
%
```

Variationer på denna funktion är funktionerna `index()` och `rindex()` som finns i `<string.h>`. De söker även efter ett tecken ur en sträng, men istället för att returnera ett index returnerar de en pekare till tecknet, eller `0` om tecknet inte kunde hittas. De returnerar alltså en pekare till den substräng som börjar där tecknet hittades. Skillnaden mellan `index()` och `rindex()` är att den förra söker från början av strängen medan den senare söker från slutet av strängen. Vi kunde således skriva om vårt program från ovan på följande sätt:

```
#include <iostream>
#include <string.h>

int main () {
    char * Found;
    char * S = "En sträng ur vilken vi söker text";

    cout << "Söksträngen är: '" << S << "'" << endl;

    // sök ett 'v'
    if ( ( Found = index (S, 'v' ) ) != 0 ) {
        cout << "Hittade ett 'v' på index " << Found - S << endl;
    }
    else {
        cout << "Hittade inget 'v'" << endl;
    }

    // sök ett 'P'
    if ( ( Found = index (S, 'P' ) ) != 0 ) {
        cout << "Hittade ett 'P' på index " << Found - S << endl;
    }
    else {
        cout << "Hittade inget 'P'" << endl;
    }
}
```

Notera att vi använder oss av pekararitmetik för att beräkna indexet i strängen. Vi vet att om tecknet hittades är det längre fram i minnet än `S`, så vi kan subtrahera `S` från `Found` utan problem för att hitta samma index som `findChar()` gav oss. Använda `rindex()` fungerar på samma sätt.

Söka substrängar

Förutom att endas söka efter enskilda tecken i en sträng kan det förekomma situationer då man behöver kontrollera om en viss sträng är en substräng av en annan sträng, och i så fall på vilket index den börjar. Konceptuellt är det ganska enkelt att göra, och vi kan använda oss av följande funktion för att utföra sökningen:

```
#include <iostream>
#include <string.h>
```

```

int findSubstring (char * String, char * Seek) {
    unsigned int Index;

    for ( Index = 0; Index <= strlen (String) - strlen (Seek); Index++ ) {
        // kanske strängen börjar här?
        if ( strcmp (String + Index, Seek, strlen (Seek)) == 0 ) {
            // vi hittade den!
            return Index;
        }
    }

    // ingen sådan sträng
    return -1;
}

int main () {
    int Index;
    char * S = "En sträng ur vilken vi söker text";

    cout << "Söksträngen är: '" << S << "'" << endl;

    // sök efter "text"
    if ( ( Index = findSubstring (S, "text" ) ) != -1 ) {
        cout << "Hittade 'text' på index " << Index << endl;
    }
    else {
        cout << "Hittade inte 'text'" << endl;
    }

    // sök efter "finns inte"
    if ( ( Index = findSubstring (S, "finns inte" ) ) != -1 ) {
        cout << "Hittade 'finns inte' på index " << Index << endl;
    }
    else {
        cout << "Hittade inte 'finns inte'" << endl;
    }
}

```

Funktionen `findSubstring()` använder sig av normala `strcmp()` för att kontrollera om substrängen `Seek` börjar någonstans i `String`. Körning av programmet ger:

```

% ./CStringFind3
Söksträngen är: 'En sträng ur vilken vi söker text'
Hittade 'text' på index 29
Hittade inte 'finns inte'
%

```

Vi kan även här använda oss av en existerande funktion ur `<string.h>` för att göra samma som `findSubstring()`. Funktionen heter `strstr()`. Vi kan då skriva om vårt program på följande sätt:

```

#include <iostream>
#include <string.h>

int main () {
    char * Found;
    char * S = "En sträng ur vilken vi söker text";

    cout << "Söksträngen är: '" << S << "'" << endl;

    // sök efter "text"
    if ( ( Found = strstr ( S, "text" ) ) != 0 ) {

```

```
        cout << "Hittade 'text' på index " << Found - S << endl;
    }
    else {
        cout << "Hittade inte 'text'" << endl;
    }

    // sök efter "finns inte"
    if ( ( Found = strstr ( S, "finns inte" ) ) != 0 ) {
        cout << "Hittade 'finns inte' på index " << Found - S << endl;
    }
    else {
        cout << "Hittade inte 'finns inte'" << endl;
    }
}
```

De parametrar som `strstr()` behöver är enligt `man`-sidan (se [avsnittet *Mera information*](#)) en "höstack" och en "nål", där man söker nålen ur höstacken. Kör man programmet ovan får man exakt samma utskrift som tidigare.

[Förutgående](#)

Konkatenering av strängar

[Hem](#)

[Upp](#)

[Nästa](#)

Mera information